SHETH L. U. J. AND SIR M. V. COLLEGE
SUBJECT: CYBERSECURITY
PRACTICAL NO.: 1

AIM: Implementing Substitution and Transposition Ciphers

1.A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

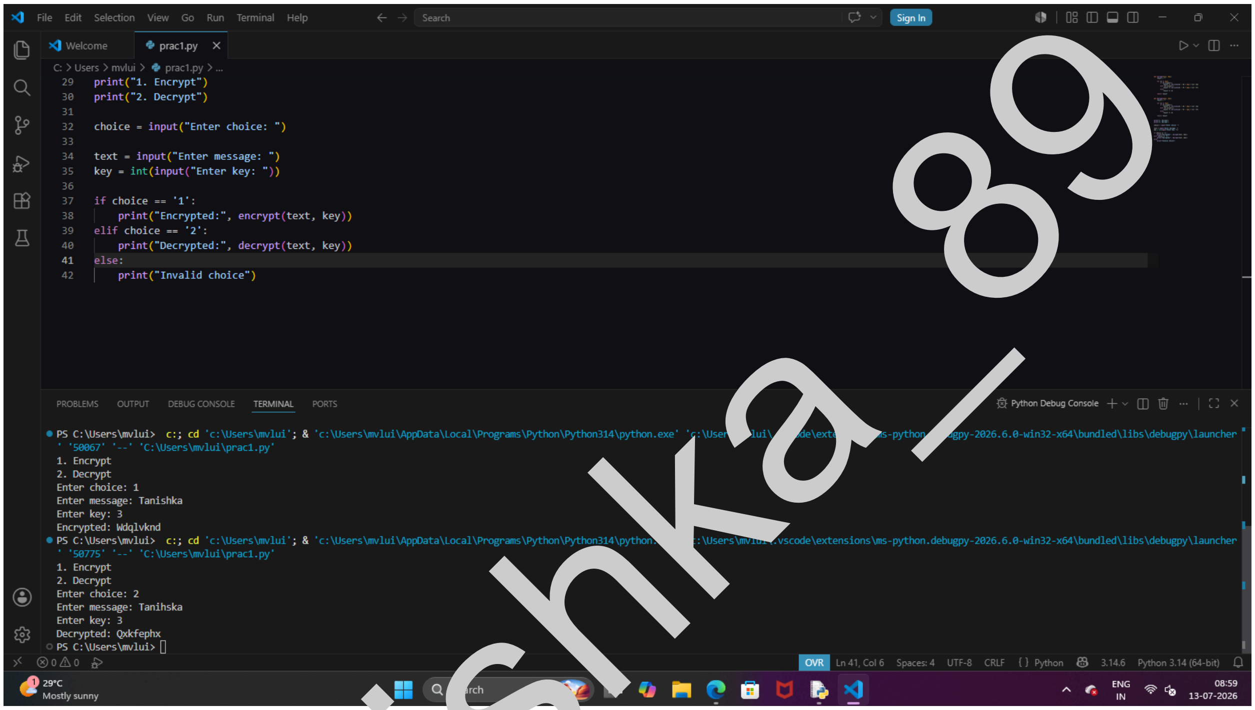
Code:-

```
def encrypt(text, key):
    result = ""

    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch

    return result


def decrypt(text, key):
    result = ""

    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 - key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 - key) % 26 + 97)
        else:
            result += ch

    return result


print("1. Encrypt")
print("2. Decrypt")

choice = input("Enter choice: ")

text = input("Enter message: ")
key = int(input("Enter key: "))

if choice == '1':
```

NAME: TANISHKA KUTWAL
ROLL NO:- T089

```
    print("Encrypted:", encrypt(text, key))
elif choice == '2':
    print("Decrypted:", decrypt(text, key))
else:
    print("Invalid choice")
```

Output:-



Gui code:-

```
import tkinter as tk
from tkinter import messagebox


# ---------------- Encryption ----------------
def encrypt():
    try:
        text = entry.get()
        key = int(key_entry.get())

        result = ""

        for ch in text:
            if ch.isupper():
                result += chr((ord(ch) - 65 + key) % 26 + 65)
            elif ch.islower():
```

```
                result += chr((ord(ch) - 97 + key) % 26 + 97)
            else:
                result += ch

        output.config(text=result)

    except ValueError:
        messagebox.showerror("Error", "Please enter a valid key.")


# ---------------- Decryption ----------------
def decrypt():
    try:
        text = entry.get()
        key = int(key_entry.get())

        result = ""

        for ch in text:
            if ch.isupper():
                result += chr((ord(ch) - 65 - key) % 26 + 65)
            elif ch.islower():
                result += chr((ord(ch) - 97 - key) % 26 + 97)
            else:
                result += ch

        output.config(text=result)

    except ValueError:
        messagebox.showerror("Error", "Please enter a valid key.")


# ---------------- Clear ----------------
def clear():
    entry.delete(0, tk.END)
    key_entry.delete(0, tk.END)
    output.config(text="")


# ---------------- Button Hover ----------------
def enter_encrypt(e):
    encrypt_btn["bg"] = "#FF1493"
```

```
def leave_encrypt(e):
    encrypt_btn["bg"] = "#FF69B4"


def enter_decrypt(e):
    decrypt_btn["bg"] = "#DB7093"


def leave_decrypt(e):
    decrypt_btn["bg"] = "#C71585"


# ---------------- Window ----------------
root = tk.Tk()
root.title("Substitution Cipher")
root.geometry("650x520")
root.configure(bg="#FFF0F5")
root.resizable(False, False)

# ---------------- Title ----------------
title = tk.Label(
    root,
    text="🔐 SUBSTITUTION CIPHER",
    font=("Segoe UI", 24, "bold"),
    bg="#FF69B4",
    fg="white",
    pady=15
)
title.pack(fill="x")

subtitle = tk.Label(
    root,
    text="Classical Caesar Cipher Encryption & Decryption",
    font=("Segoe UI", 11),
    bg="#FFF0F5",
    fg="#C71585"
)
subtitle.pack(pady=10)

# ---------------- Card ----------------
card = tk.Frame(
    root,
```

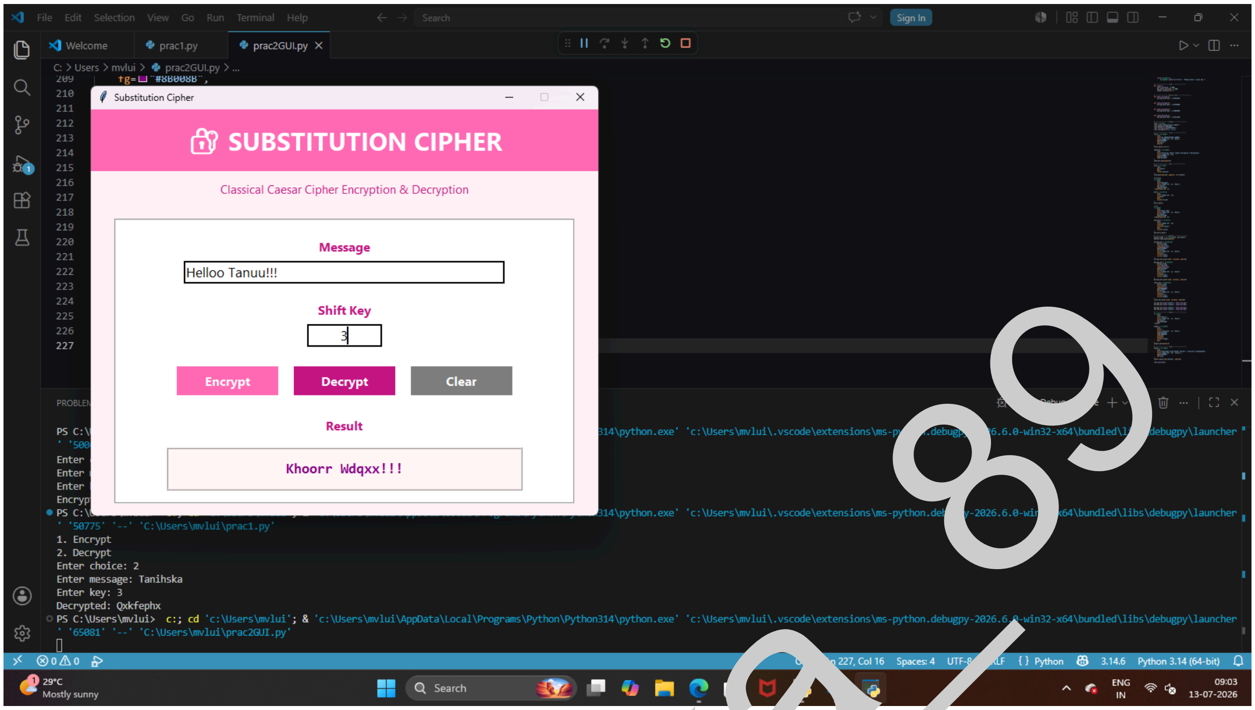
```
    bg="white",
    bd=2,
    relief="groove"
)
card.pack(padx=30, pady=15, fill="both")

# Message
tk.Label(
    card,
    text="Message",
    font=("Segoe UI", 12, "bold"),
    bg="white",
    fg="#C71585"
).pack(pady=(20, 5))

entry = tk.Entry(
    card,
    font=("Segoe UI", 13),
    width=45,
    bd=2,
    relief="solid"
)
entry.pack()

# Key
tk.Label(
    card,
    text="Shift Key",
    font=("Segoe UI", 12, "bold"),
    bg="white",
    fg="#C71585"
).pack(pady=(20, 5))

key_entry = tk.Entry(
    card,
    font=("Segoe UI", 13),
    width=10,
    justify="center",
    bd=2,
    relief="solid"
)
key_entry.pack()
```

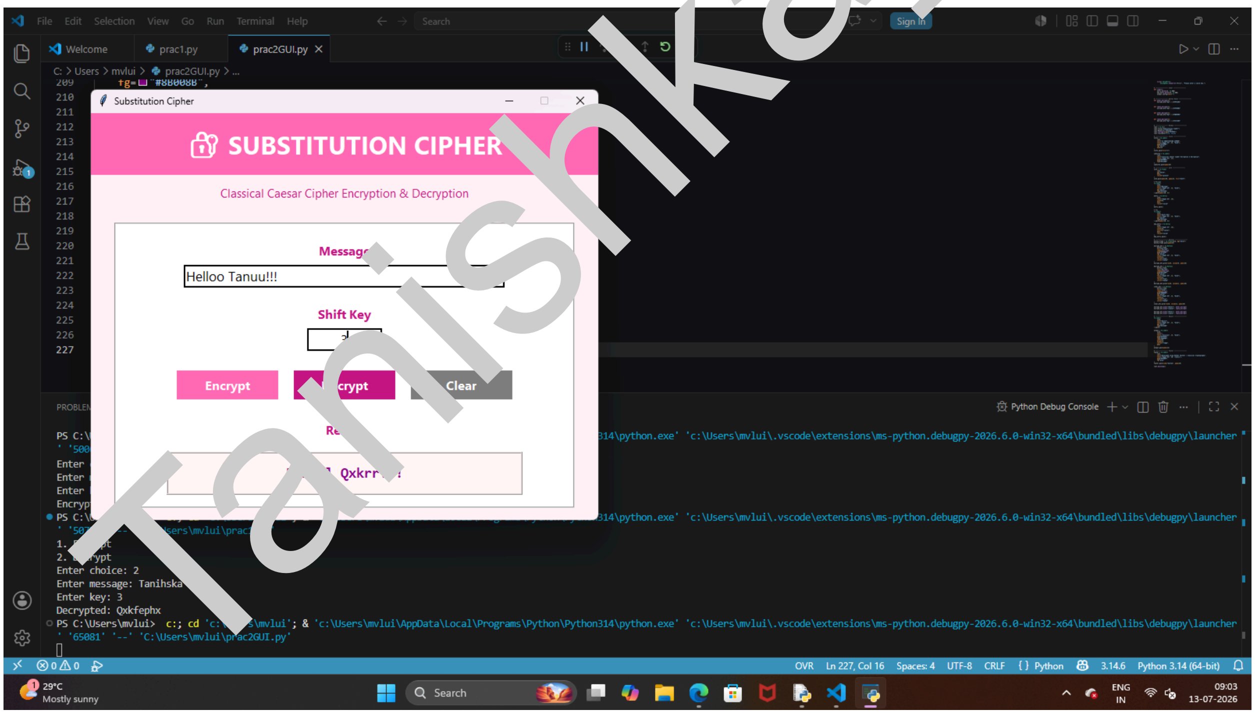
```
# ---------------- Buttons ----------------
button_frame = tk.Frame(card, bg="white")
button_frame.pack(pady=25)

encrypt_btn = tk.Button(
    button_frame,
    text="Encrypt",
    command=encrypt,
    bg="#FF69B4",
    fg="white",
    font=("Segoe UI", 12, "bold"),
    width=12,
    relief="flat",
    cursor="hand2"
)
encrypt_btn.grid(row=0, column=0, padx=10)

decrypt_btn = tk.Button(
    button_frame,
    text="Decrypt",
    command=decrypt,
    bg="#C71585",
    fg="white",
    font=("Segoe UI", 12, "bold"),
    width=12,
    relief="flat",
    cursor="hand2"
)
decrypt_btn.grid(row=0, column=1, padx=10)

clear_btn = tk.Button(
    button_frame,
    text="Clear",
    command=clear,
    bg="#808080",
    fg="white",
    font=("Segoe UI", 12, "bold"),
    width=12,
    relief="flat",
    cursor="hand2"
)
clear_btn.grid(row=0, column=2, padx=10)
```

```
encrypt_btn.bind("<Enter>", enter_encrypt)
encrypt_btn.bind("<Leave>", leave_encrypt)

decrypt_btn.bind("<Enter>", enter_decrypt)
decrypt_btn.bind("<Leave>", leave_decrypt)

# ---------------- Result ----------------
tk.Label(
    card,
    text="Result",
    font=("Segoe UI", 12, "bold"),
    bg="white",
    fg="#C71585"
).pack()

output = tk.Label(
    card,
    text="",
    font=("Consolas", 14, "bold"),
    bg="#FFF5F8",
    fg="#8B008B",
    width=45,
    height=3,
    relief="ridge",
    bd=2
)
output.pack(pady=15)

# ---------------- Footer ----------------
footer = tk.Label(
    root,
    text="Developed using Python Tkinter | Classical Cryptography",
    font=("Segoe UI", 10, "italic"),
    bg="#FFF0F5",
    fg="gray"
)
footer.pack(side="bottom", pady=10)

root.mainloop()
```

Output:-

SHETH L. U. J. AND SIR M. V. COLLEGE
SUBJECT: CYBERSECURITY
PRACTICAL NO.: 1





NAME: TANISHKA KUTWAL
ROLL NO:- T089

SHETH L. U. J. AND SIR M. V. COLLEGE
SUBJECT: CYBERSECURITY
PRACTICAL NO.: 1

1. B Design and implement algorithms to encrypt and decrypt messages using transposition techniques

Code:

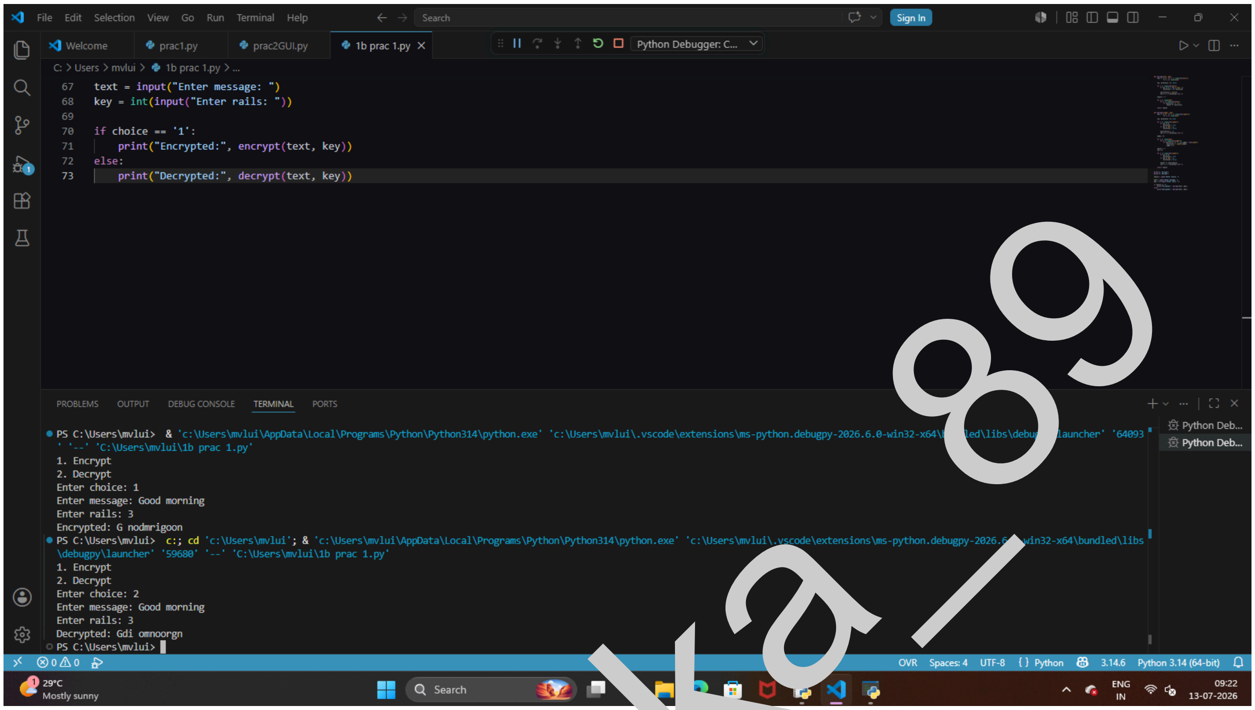
```
def encrypt(text, key):
    rail = [['\n' for i in range(len(text))]
            for j in range(key)]

    row, direction = 0, False

    for i in range(len(text)):
        if row == 0 or row == key - 1:
            direction = not direction

        rail[row][i] = text[i]
        row += 1 if direction else -1

    result = ""

    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                result += rail[i][j]

    return result


def decrypt(cipher, key):
    rail = [['\n' for i in range(len(cipher))]
            for j in range(key)]

    row, direction = 0, None

    for i in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False
```


NAME: TANISHKA KUTWAL
ROLL NO:- T089

```
        rail[row][i] = '*'
        row += 1 if direction else -1

    index = 0

    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row = 0

    for i in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False

        result += rail[row][i]
        row += 1 if direction else -1

    return result


print("1. Encrypt")
print("2. Decrypt")

choice = input("Enter choice: ")

text = input("Enter message: ")
key = int(input("Enter rails: "))

if choice == '1':
    print("Encrypted:", encrypt(text, key))
else:
    print("Decrypted:", decrypt(text, key))
```

SHETH L. U. J. AND SIR M. V. COLLEGE
SUBJECT: CYBERSECURITY
PRACTICAL NO.: 1

Output:



GUI code:-

```
import tkinter as tk
from tkinter import messagebox

# -------------------- Rail Fence Encryption --------------------
def encrypt(text, key):
    if key <= 1:
        return text

    rail = [['\n' for _ in range(len(text))] for _ in range(key)]

    row = 0
    direction_down = False

    for i in range(len(text)):
        if row == 0 or row == key - 1:
            direction_down = not direction_down

        rail[row][i] = text[i]
        row += 1 if direction_down else -1
```


NAME: TANISHKA KUTWAL
ROLL NO:- T089

```
    result = ""

    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                result += rail[i][j]

    return result


# -------------------- Rail Fence Decryption --------------------
def decrypt(cipher, key):
    if key <= 1:
        return cipher

    rail = [['\n' for _ in range(len(cipher))] for _ in range(key)]

    row = 0
    direction_down = None

    for i in range(len(cipher)):
        if row == 0:
            direction_down = True
        if row == key - 1:
            direction_down = False

        rail[row][i] = '*'
        row += 1 if direction_down else -1

    index = 0

    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row = 0
```

```
    for i in range(len(cipher)):
        if row == 0:
            direction_down = True
        if row == key - 1:
            direction_down = False

        result += rail[row][i]
        row += 1 if direction_down else -1

    return result


# -------------------- Button Functions --------------------
def encrypt_message():
    try:
        text = message_entry.get()

        if text == "":
            messagebox.showwarning("Warning", "Please enter a message.")
            return

        key = int(key_entry.get())

        if key < 2:
            messagebox.showwarning("Warning", "Number of rails must be at least 2.")
            return

        result = encrypt(text, key)
        output.config(text=result)

    except ValueError:
        messagebox.showerror("Error", "Please enter a valid number.")


def decrypt_message():
    try:
        text = message_entry.get()

        if text == "":
```

```
            messagebox.showwarning("Warning", "Please enter a message.")
            return

        key = int(key_entry.get())

        if key < 2:
            messagebox.showwarning("Warning", "Number of rails must be at least 2.")
            return

        result = decrypt(text, key)
        output.config(text=result)

    except ValueError:
        messagebox.showerror("Error", "Please enter a valid number.")


def clear_fields():
    message_entry.delete(0, tk.END)
    key_entry.delete(0, tk.END)
    output.config(text="")


# -------------------- Main Window --------------------
root = tk.Tk()
root.title("Rail Fence Cipher")
root.geometry("650x500")
root.configure(bg="#EAF6FF")
root.resizable(False, False)

# -------------------- Heading --------------------
heading = tk.Label(
    root,
    text="🔐 RAIL FENCE CIPHER",
    font=("Segoe UI", 22, "bold"),
    bg="#007ACC",
    fg="white",
    pady=15
)
heading.pack(fill="x")
```

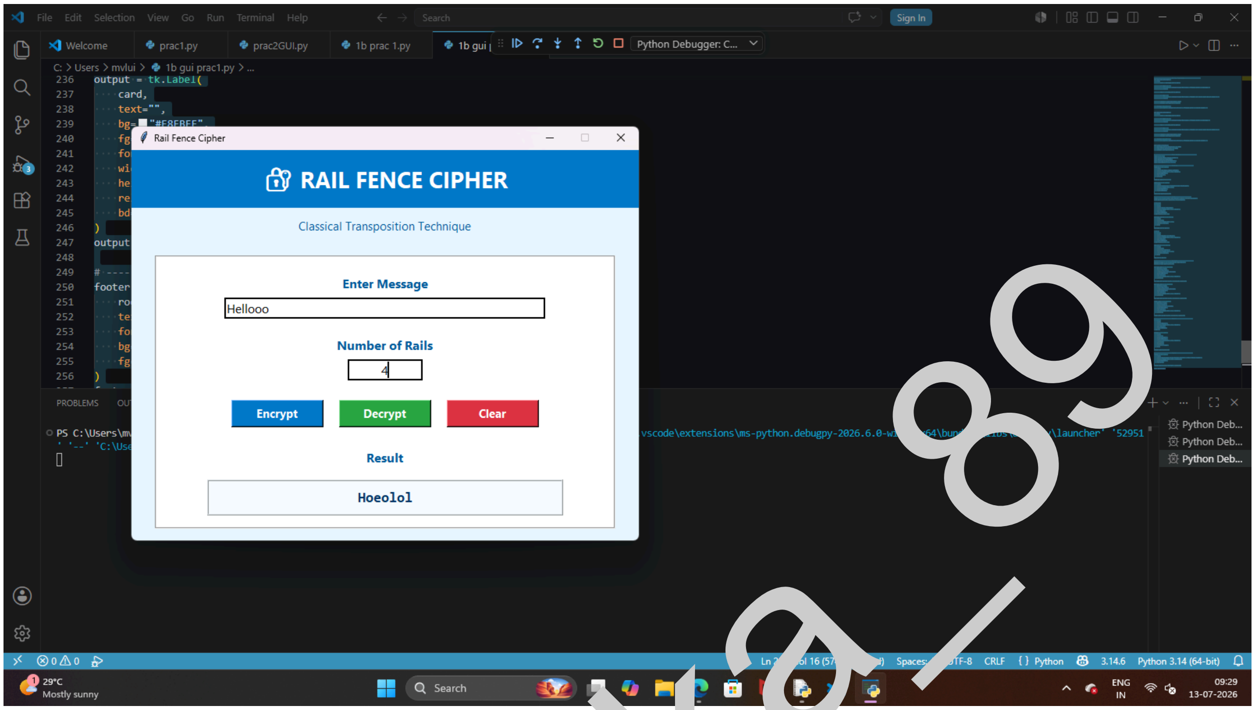
```
subtitle = tk.Label(
    root,
    text="Classical Transposition Technique",
    font=("Segoe UI", 11),
    bg="#EAF6FF",
    fg="#005A9C"
)
subtitle.pack(pady=10)

# -------------------- Card --------------------
card = tk.Frame(root, bg="white", bd=2, relief="groove")
card.pack(padx=30, pady=15, fill="both")

# Message
tk.Label(
    card,
    text="Enter Message",
    font=("Segoe UI", 12, "bold"),
    bg="white",
    fg="#005A9C"
).pack(pady=(20, 5))

message_entry = tk.Entry(
    card,
    width=45,
    font=("Segoe UI", 12),
    relief="solid",
    bd=2
)
message_entry.pack()

# Rails
tk.Label(
    card,
    text="Number of Rails",
    font=("Segoe UI", 12, "bold"),
    bg="white",
    fg="#005A9C"
).pack(pady=(20, 5))
```

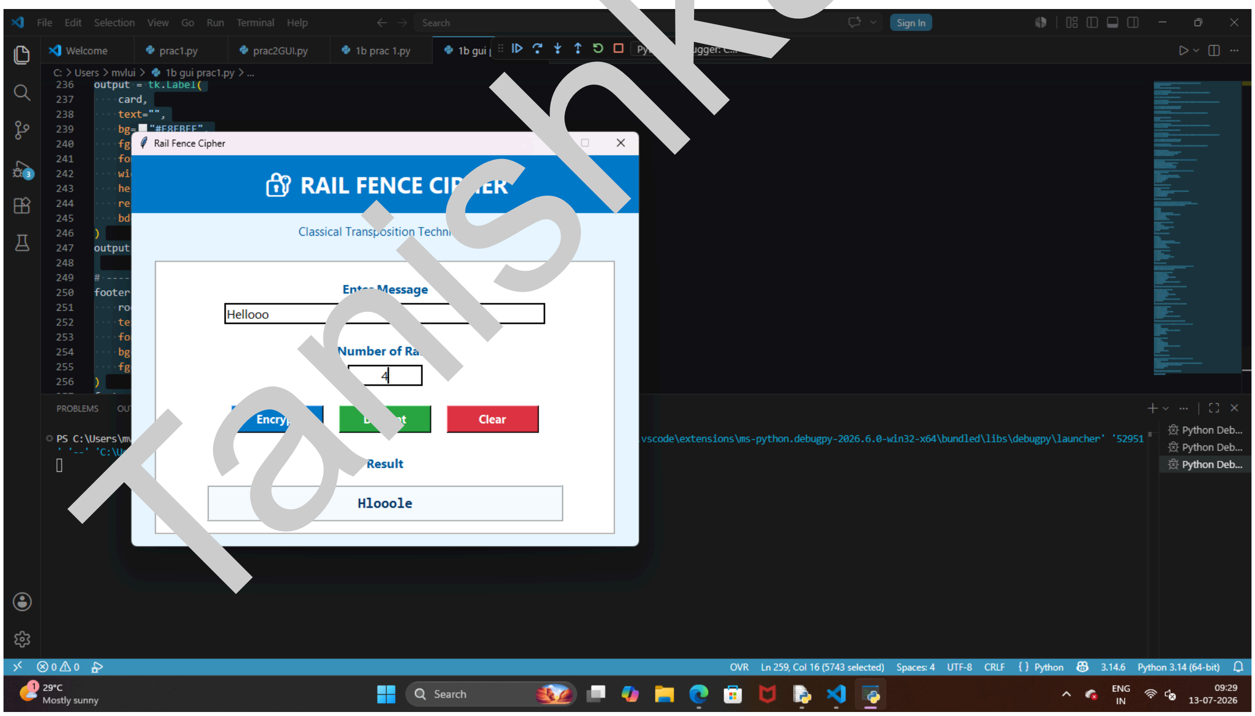
```
key_entry = tk.Entry(
    card,
    width=10,
    justify="center",
    font=("Segoe UI", 12),
    relief="solid",
    bd=2
)
key_entry.pack()

# -------------------- Buttons --------------------
button_frame = tk.Frame(card, bg="white")
button_frame.pack(pady=25)

encrypt_btn = tk.Button(
    button_frame,
    text="Encrypt",
    command=encrypt_message,
    bg="#007ACC",
    fg="white",
    font=("Segoe UI", 11, "bold"),
    width=12
)
encrypt_btn.grid(row=0, column=0, padx=10)

decrypt_btn = tk.Button(
    button_frame,
    text="Decrypt",
    command=decrypt_message,
    bg="#28A745",
    fg="white",
    font=("Segoe UI", 11, "bold"),
    width=12
)
decrypt_btn.grid(row=0, column=1, padx=10)

clear_btn = tk.Button(
    button_frame,
    text="Clear",
    command=clear_fields,
```

```
    bg="#DC3545",
    fg="white",
    font=("Segoe UI", 11, "bold"),
    width=12
)
clear_btn.grid(row=0, column=2, padx=10)

# -------------------- Result --------------------
tk.Label(
    card,
    text="Result",
    font=("Segoe UI", 12, "bold"),
    bg="white",
    fg="#005A9C"
).pack()

output = tk.Label(
    card,
    text="",
    bg="#F8FBFF",
    fg="#003366",
    font=("Consolas", 14, "bold"),
    width=45,
    height=3,
    relief="ridge",
    bd=2
)
output.pack(pady=1[illegible])

# -------------------- Footer --------------------
footer = tk.Label(
    root,
    text="Developed using Python Tkinter | Rail Fence Cipher",
    font=("Segoe UI", 10, "italic"),
    bg="#EAF6FF",
    fg="gray"
)
footer.pack(side="bottom", pady=10)

root.mainloop()
```

SHETH L. U. J. AND SIR M. V. COLLEGE
SUBJECT: CYBERSECURITY
PRACTICAL NO.: 1

Output:-





NAME: TANISHKA KUTWAL
ROLL NO:- T089